BÀI THỰC HÀNH CTDL
NGĂN XẾP (STACK) VÀ HÀNG ĐỢI (QUEUE)

Trong bài thực hành này chúng ta sẽ tiếp tục hành trình khám phá các cấu trúc dữ liệu nền tảng trong Python với hai cấu trúc rất đặc biệt: Stack và Queue.

Nếu List và LinkedList cho phép chúng ta truy cập tự do, thì Stack và Queue lại có những quy tắc truy cập rất nghiêm ngặt. Chính sự giới hạn này lại tạo nên sức mạnh của chúng trong việc giải quyết nhiều lớp bài toán cụ thể.

- **Stack (Ngăn xếp):** Hoạt động theo nguyên tắc LIFO (Last-In, First-Out - Vào sau, Ra trước). Hãy tưởng tượng một chồng đĩa, bạn chỉ có thể đặt một chiếc đĩa mới lên trên cùng và khi lấy ra cũng chỉ có thể lấy chiếc trên cùng.
- **Queue (Hàng đợi):** Hoạt động theo nguyên tắc FIFO (First-In, First-Out - Vào trước, Ra trước). Giống hệt như khi bạn xếp hàng ở quầy thanh toán, ai đến trước sẽ được phục vụ trước.

Mục Đích và Yêu Cầu Chung

🎯 Mục đích

- **Hiểu rõ nguyên tắc LIFO và FIFO:** Nắm vững và phân biệt được sự khác nhau cốt lõi giữa Stack và Queue.
- **Thành thạo các phương thức chính:** Sử dụng thành thạo Push, Pop, Peek của Stack và Enqueue, Dequeue, Peek của Queue.
- **Nhận diện bài toán:** Rèn luyện khả năng phân tích một vấn đề và xác định được nên sử dụng Stack hay Queue để giải quyết một cách tối ưu.
- **Ứng dụng thực tế:** Liên hệ kiến thức được học vào các bài toán thực tế như quản lý tiến trình, phân tích cú pháp, duyệt thuật toán...

📋 Yêu cầu chung cho tất cả bài tập

- **Môi trường:** Viết ứng dụng dạng Console Application bằng **Python**.
- **Code sạch:** Viết code rõ ràng, dễ đọc, tuân thủ các quy ước đặt tên.
- **Xử lý ngoại lệ:** Chú ý xử lý các trường hợp khi cố gắng Pop hoặc Dequeue từ một collection rỗng.
- **Học hỏi chủ động:** Sử dụng AI như một người trợ giảng, đặt câu hỏi để đào sâu vấn đề, không chỉ sao chép code.

Bài tập 1: Đảo ngược một chuỗi ký tự - Mức độ: Dễ (Stack)

Đây là bài tập "Hello World!" của Stack, giúp bạn hình dung rõ nhất về nguyên tắc LIFO.

Yêu cầu chi tiết:

- Viết một chương trình yêu cầu người dùng nhập vào một chuỗi (ví dụ: "hello").
- Sử dụng một Stack để lưu trữ từng ký tự của chuỗi.
- Lấy các ký tự ra khỏi Stack để tạo thành một chuỗi mới.
- In chuỗi mới ra màn hình. Kết quả phải là chuỗi đảo ngược (ví dụ: "olleh").

💡 Gợi ý:

- Dùng vòng lặp `for` để duyệt qua chuỗi đầu vào và `append()` (tương đương Push) từng ký tự vào danh sách đóng vai trò Stack.
- Dùng vòng lặp `while` với điều kiện `len(stack) > 0`. Bên trong vòng lặp, dùng `pop()` để lấy ký tự trên cùng ra và đưa vào danh sách kết quả.

💻 Code mẫu:

```
def main():
    original_string = input("Nhập một chuỗi để đảo ngược: ")
    char_stack = []

    # 1. Đẩy (Push) từng ký tự vào Stack
    for c in original_string:
        char_stack.append(c)

    # 2. Lấy ra (Pop) để xây dựng chuỗi mới
    reversed_chars = []
    while len(char_stack) > 0:
        reversed_chars.append(char_stack.pop())

    reversed_string = "".join(reversed_chars)
    print("Chuỗi sau khi đảo ngược:", reversed_string)

if __name__ == "__main__":
    main()
```

🤖 Vận dụng AI:

- **Hỏi để so sánh:** Hỏi AI: *"Tại sao Stack lại là lựa chọn tự nhiên cho bài toán này? Điều gì sẽ xảy ra nếu tôi cố gắng giải bài này bằng Queue?"*
- **Tối ưu hóa:** Hỏi AI: *"Trong Python, tại sao gom các ký tự vào list rồi dùng `''.join()` lại tối ưu hiệu năng hơn việc cộng dồn chuỗi liên tục bằng toán tử `+`?"*

Bài tập 2: Cơ chế Hoàn tác & Làm lại (Undo / Redo) - Mức độ: Dễ - Trung bình (2 Stacks)

Bài tập này mô phỏng tính năng `Ctrl + Z` (Undo) và `Ctrl + Y` (Redo) trong các trình soạn thảo văn bản như Word, VS Code.

Yêu cầu chi tiết:

- Viết một trình quản lý văn bản đơn giản với hai Stack: `undo_stack` và `redo_stack`.
- Xây dựng menu chức năng:
    1. **Nhập văn bản mới (Type):** Thêm một chuỗi ký tự vào cuối văn bản hiện tại. Lưu trạng thái trước đó vào `undo_stack` và xóa sạch `redo_stack`.
    2. **Hoàn tác (Undo):** Hủy thao tác vừa nhập, quay lại trạng thái trước đó, đồng thời đẩy trạng thái bị hủy vào `redo_stack`.
    3. **Làm lại (Redo):** Phục hồi lại thao tác vừa bị Undo.
    4. **Xem văn bản hiện tại.**
    5. **Thoát.**

Gợi ý:

- Dùng hai danh sách Python làm Stack: `undo_stack = []` và `redo_stack = []`.
- Khi gọi `undo`: kiểm tra `len(undo_stack) > 0`, đẩy trạng thái hiện tại vào `redo_stack`, sau đó gán trạng thái hiện tại bằng phần tử vừa lấy từ `undo_stack.pop()`.
- Khi gọi `redo`: kiểm tra `len(redo_stack) > 0`, đẩy trạng thái hiện tại vào `undo_stack`, rồi cập nhật trạng thái hiện tại từ `redo_stack.pop()`.

Code mẫu:

```
class TextEditor:
    def __init__(self):
        self.text = ""
        self.undo_stack = []
        self.redo_stack = []

    def write(self, new_text: str):
        # Lưu trạng thái cũ trước khi thay đổi
        self.undo_stack.append(self.text)
        self.redo_stack.clear()  # Nhập mới sẽ làm mất lịch sử Redo
        self.text += new_text
        print(f'-> Đã thêm: "{new_text}"')

    def undo(self):
        if not self.undo_stack:
            print("Không còn thao tác nào để Undo.")
            return
        self.redo_stack.append(self.text)
        self.text = self.undo_stack.pop()
        print("-> Đã hoàn tác (Undo).")

    def redo(self):
        if not self.redo_stack:
            print("Không còn thao tác nào để Redo.")
            return
        self.undo_stack.append(self.text)
        self.text = self.redo_stack.pop()
        print("-> Đã làm lại (Redo).")

    def show(self):
        print(f'Nội dung hiện tại: "{self.text}"')
```

🤖 Vận dụng AI:

- **Tối ưu bộ nhớ:** Hỏi AI: *"Lưu toàn bộ chuỗi ký tự trong mỗi lần Undo rất tốn RAM nếu chuỗi dài hàng MB. Hãy gợi ý cách dùng Design Pattern Command kết hợp Stack để chỉ lưu thao tác ngược (nghịch đảo) thay vì sao chép toàn bộ văn bản."*

Bài tập 3: Mô phỏng hàng đợi in ấn - Mức độ: Dễ (Queue)

Bài tập này mô phỏng một ứng dụng rất thực tế của Queue: quản lý các tác vụ cần thực hiện tuần tự.

Yêu cầu chi tiết:

Tạo một Queue để lưu tên các tài liệu đang chờ được in (ví dụ: "TaiLieuA.doc", "BaoCaoThang9.pdf").

Xây dựng menu chức năng:

1. Thêm tài liệu mới vào hàng đợi in.
2. Thực hiện in tài liệu tiếp theo (lấy tài liệu đầu hàng đợi ra và thông báo đã in xong).
3. Xem tài liệu sắp được in (xem tài liệu đầu hàng đợi nhưng không xóa nó).
4. Xem toàn bộ hàng đợi.
5. Thoát.

💡 Gợi ý:

- Sử dụng `collections.deque` để tạo Queue chuẩn có độ phức tạp $O(1)$ ở hai đầu.
- **Thêm:** Dùng `.append()` (tương đương Enqueue) để thêm tài liệu vào cuối hàng đợi.
- **In:** Dùng `.popleft()` (tương đương Dequeue) để lấy ra và xóa tài liệu đầu hàng đợi.
- **Xem:** Dùng chỉ số `queue[0]` (tương đương Peek) để xem tài liệu đầu tiên mà không xóa.
- Luôn kiểm tra `len(queue) > 0` trước khi `popleft()` hoặc truy cập phần tử đầu để tránh lỗi.

💻 Code mẫu:

```
from collections import deque

def main():
    print_queue = deque()

    # Phần menu tương tự các bài trước, đây là các logic chức năng cốt lõi
    # 1. Thêm tài liệu
    # print_queue.append("TaiLieuA.doc")

    # 2. In tài liệu
    if len(print_queue) > 0:
        document_to_print = print_queue.popleft()
```

```
        print(f"Đang in: {document_to_print}... Đã in xong!")
    else:
        print("Không có tài liệu nào trong hàng đợi.")

    # 3. Xem tài liệu sắp in (Peek)
    if len(print_queue) > 0:
        print(f"Tài liệu tiếp theo sẽ được in là: {print_queue[0]}")

if __name__ == "__main__":
    main()
```

🤖 Vận dụng AI:

- **Hiểu sâu về phương thức:** Hỏi AI: *"So sánh sự khác biệt giữa Dequeue (popleft) và Peek (index 0) trong Queue. Cho ví dụ về trường hợp nên dùng Peek()."*
- **Mở rộng vấn đề:** Yêu cầu AI: *"Hãy giúp tôi mở rộng chương trình này. Tôi muốn mỗi tài liệu có thêm độ ưu tiên (priority). Hệ thống sẽ luôn in tài liệu có độ ưu tiên cao nhất trước. Tôi nên dùng cấu trúc dữ liệu nào trong Python thay cho Queue?" (AI sẽ giới thiệu bạn về `heapq` hoặc `queue.PriorityQueue`).*

Bài tập 4: Mô phỏng hàng chờ tổng đài chăm sóc khách hàng - Mức độ: Dễ - Trung bình (Queue)

Mô phỏng hệ thống phân phối cuộc gọi đến nhân viên chăm sóc khách hàng theo quy luật FIFO.

Yêu cầu chi tiết:

- Tạo lớp `Call` gồm: `call_id` (mã cuộc gọi), `caller_name` (tên người gọi), `wait_time` (thời điểm gọi đến).
- Tạo một hàng đợi lưu các cuộc gọi đang chờ hỗ trợ.
- Viết chương trình thực hiện:
    1. Thêm cuộc gọi mới vào hàng chờ.
    2. Nhân viên tiếp nhận: Lấy cuộc gọi đầu tiên ra khỏi hàng đợi để xử lý.
    3. Kiểm tra số lượng người còn chờ và thông báo người phải chờ lâu nhất hiện tại (không xóa họ khỏi hàng đợi).

💡 Gợi ý:

- Dùng `collections.deque` để quản lý hàng đợi nhằm đảm bảo tốc độ nhanh nhất khi tiếp nhận.
- Dùng `queue[0]` để xem thông tin cuộc gọi đang chờ lâu nhất (Peek).

💻 Code mẫu:

```
from collections import deque
from dataclasses import dataclass
import time
```

```
@dataclass
class Call:
    call_id: int
    caller_name: str
    arrival_time: float

def main():
    call_queue = deque()
    call_id_counter = 1

    # 1. Thêm khách hàng gọi đến
    def receive_call(name: str):
        nonlocal call_id_counter
        new_call = Call(call_id_counter, name, time.time())
        call_queue.append(new_call)
        print(f"Cuộc gọi #{call_id_counter} từ '{name}' đã vào hàng chờ.")
        call_id_counter += 1

    # 2. Xử lý cuộc gọi
    def answer_call():
        if not call_queue:
            print("Tổng đài đang rảnh, không có cuộc gọi chờ.")
            return
        call = call_queue.popleft()
        waited_seconds = round(time.time() - call.arrival_time, 2)
        print(f"-> Đang kết nối cuộc gọi #{call.call_id}
('{call.caller_name}'). Thời gian chờ: {waited_seconds}s.")

    # Minh họa kịch bản
    receive_call("Nguyễn Văn A")
    receive_call("Trần Thị B")
    time.sleep(1)
    receive_call("Lê Văn C")

    print(f"\nHiện có {len(call_queue)} người đang chờ.")
    print(f"Người chờ tiếp theo sẽ là: {call_queue[0].caller_name}\n")

    answer_call()
    answer_call()

if __name__ == "__main__":
    main()
```

🤖 **Vận dụng AI:**

- **Mở rộng kịch bản:** Yêu cầu AI: *"Nếu khách hàng VIP được ưu tiên nghe máy trước khách hàng thường dù họ gọi đến sau, hãy viết lại chương trình bằng cấu trúc dữ liệu PriorityQueue trong Python."*

Bài tập 5: Kiểm tra chuỗi ngoặc hợp lệ - Mức độ: Trung bình (Stack)

Đây là một bài toán kinh điển trong khoa học máy tính và thường xuất hiện trong các buổi phỏng vấn, dùng để kiểm tra tính hợp lệ của một biểu thức.

Yêu cầu chi tiết:

- Viết một hàm nhận đầu vào là một chuỗi chỉ chứa các ký tự `(`, `)`, `{`, `}`, `[`, `]`.
- Hàm này sẽ trả về `True` nếu chuỗi ngoặc là hợp lệ, và `False` nếu không.
- Một chuỗi ngoặc hợp lệ phải thỏa mãn 2 điều kiện:
    1. Mỗi ngoặc mở phải được đóng bởi cùng loại ngoặc.
    2. Các ngoặc phải được đóng theo đúng thứ tự. Ví dụ: `()[]` là hợp lệ, nhưng `([)]` là không hợp lệ.

💡 Gợi ý:

- Duyệt qua chuỗi đầu vào.
- Nếu gặp một ngoặc mở (`(`, `[`, `{`), hãy Push nó vào Stack (`append()`).
- Nếu gặp một ngoặc đóng (`)`, `]`, `}`), hãy kiểm tra:
    - Nếu Stack rỗng --> chuỗi không hợp lệ (vì có ngoặc đóng mà không có ngoặc mở).
    - `pop()` ngoặc mở ra khỏi Stack và so sánh. Nếu nó không khớp với ngoặc đóng hiện tại --> chuỗi không hợp lệ.
- Sau khi duyệt hết chuỗi, nếu Stack vẫn còn phần tử --> chuỗi không hợp lệ (vì có ngoặc mở mà không có ngoặc đóng tương ứng).
- Nếu Stack rỗng --> chuỗi hợp lệ.

💻 Code mẫu:

```
class Solution:
    def is_valid(self, s: str) -> bool:
        stack = []
        bracket_pairs = {
            '(': ')',
            '[': ']',
            '{': '}'
        }

        for c in s:
            if c in bracket_pairs:  # Nếu là ngoặc mở
                stack.append(c)
            else:  # Nếu là ngoặc đóng
                if len(stack) == 0:
                    return False  # Không có ngoặc mở tương ứng
                open_bracket = stack.pop()
                if bracket_pairs[open_bracket] != c:
                    return False  # Không khớp loại ngoặc

        return len(stack) == 0  # Hợp lệ nếu stack rỗng cuối cùng

# Kiểm thử
solution = Solution()
```

```
print(solution.is_valid("()[]{}"))  # True
print(solution.is_valid("([)]"))    # False
```

🤖 Vận dụng AI:

- **Phân tích logic:** Hỏi AI: *"Giải thích tại sao nguyên tắc LIFO của Stack lại là chìa khóa để giải quyết bài toán kiểm tra chuỗi ngoặc hợp lệ."*
- **Tìm trường hợp lỗi:** Yêu cầu AI: *"Hãy cho tôi 5 chuỗi đầu vào khác nhau dùng để kiểm thử hàm is_valid, bao gồm cả trường hợp đúng và các trường hợp sai phổ biến."*

Bài tập 6: Tìm phần tử lớn hơn tiếp theo (Next Greater Element) - Mức độ: Khó (Monotonic Stack)

Đây là bài toán phỏng vấn kinh điển áp dụng kỹ thuật **Monotonic Stack (Ngăn xếp đơn điệu)** để tối ưu ngăn xếp

Yêu cầu chi tiết:

- Cho một mảng số nguyên `arr`. Với mỗi phần tử, hãy tìm phần tử đầu tiên nằm bên phải nó có giá trị lớn hơn nó. Nếu không có phần tử nào lớn hơn, trả về `-1`.
- *Ví dụ:* `arr = [4, 5, 2, 25]` --> Kết quả: `[5, 25, 25, -1]`.

💡 Gợi ý:

- Sử dụng một Stack lưu **chỉ số (indices)** của các phần tử trong mảng.
- Duyệt qua mảng: chừng nào Stack còn phần tử và giá trị hiện tại lớn hơn phần tử ở đỉnh Stack:
    - Lấy chỉ số ra khỏi Stack và gán kết quả của vị trí đó bằng giá trị hiện tại.
- Đẩy chỉ số hiện tại vào Stack. Các phần tử còn lại trong Stack sau khi duyệt xong sẽ mang giá trị `-1`.

💻 Code mẫu:

```
def next_greater_element(nums: list[int]) -> list[int]:
    n = len(nums)
    result = [-1] * n
    stack = []  # Lưu chỉ số (indices)

    for i in range(n):
        # Khi tìm thấy phần tử lớn hơn phần tử ở đỉnh stack
        while stack and nums[i] > nums[stack[-1]]:
            prev_index = stack.pop()
            result[prev_index] = nums[i]

        stack.append(i)

    return result
```

```
def main():
    arr = [4, 5, 2, 25]
    res = next_greater_element(arr)
    print("Mảng gốc:                    ", arr)
    print("Phần tử lớn hơn tiếp theo:  ", res)

if __name__ == "__main__":
    main()
```

🤖 Vận dụng AI:

- **So sánh độ phức tạp:** Hỏi AI: *"Tại sao thuật toán dùng Monotonic Stack trên chỉ mất thời gian hơn so với Stack thông thường mặc dù có một vòng lặp `while` nằm bên trong vòng lặp `for`?"* (AI sẽ giải thích phương pháp phân tích chi phí khấu hao - Amortized Analysis).

Bài tập 7: Mô phỏng quá trình tải trang web (Web Page Loading) - Mức độ: Khó (Queue)

Bài tập này mô phỏng cách một trình duyệt web có thể dùng Queue để quản lý việc tải các tài nguyên (hình ảnh, script, css) của một trang web theo thứ tự.

Yêu cầu chi tiết:

- Tạo một lớp `Resource` có thuộc tính `name` (string) và `size` (int, giả sử là KB).
- Tạo một Queue để chứa các tài nguyên cần tải.
- Viết chương trình thực hiện các bước sau:
    - Thêm một vài tài nguyên vào Queue (ví dụ: "style.css", "logo.png", "main.js").
    - Bắt đầu quá trình "tải": Lần lượt Dequeue từng tài nguyên, giả lập thời gian tải bằng cách cho chương trình dừng một khoảng thời gian tỷ lệ với `size` (dùng `time.sleep()`).
    - In ra thông báo mỗi khi bắt đầu tải và khi tải xong một tài nguyên.

💡 Gợi ý:

- Tạo class `Resource`: dùng `dataclass` hoặc `__init__` thông thường.
- Khởi tạo Queue bằng `collections.deque` và `.append()` vài đối tượng `Resource`.
- Sử dụng vòng lặp `while len(loading_queue) > 0`.
- Trong vòng lặp, `popleft()` tài nguyên ra, in thông báo bắt đầu, dùng `time.sleep(resource.size * 0.01)` (ví dụ, 10ms cho mỗi KB = 0.01 giây), rồi in thông báo hoàn thành.

💻 Code mẫu:

```
import time
from collections import deque
```

```
from dataclasses import dataclass

@dataclass
class Resource:
    name: str
    size: int  # tính bằng KB

def main():
    loading_queue = deque()
    loading_queue.append(Resource(name="style.css", size=150))
    loading_queue.append(Resource(name="logo.png", size=50))
    loading_queue.append(Resource(name="main.js", size=300))
    loading_queue.append(Resource(name="jquery.js", size=90))

    print("Bắt đầu tải tài nguyên trang web...")
    total_size = 0

    while len(loading_queue) > 0:
        current_resource = loading_queue.popleft()
        print(f" -> Đang tải {current_resource.name}
({current_resource.size}KB)...", end="", flush=True)

        # Giả lập thời gian tải (10ms = 0.01 giây cho mỗi KB)
        time.sleep(current_resource.size * 0.01)
        print(" Xong!")
        total_size += current_resource.size

    print(f"\nTải trang hoàn tất! Tổng dung lượng: {total_size}KB.")

if __name__ == "__main__":
    main()
```

🤖 Vận dụng AI:

- **Tìm hiểu khái niệm liên quan:** Hỏi AI: *"Bài toán này có liên quan gì đến thuật toán duyệt đồ thị theo chiều rộng (BFS - Breadth-First Search)? Tại sao BFS lại dùng Queue?"*
- **Thách thức nâng cao:** Yêu cầu AI: *"Bây giờ, hãy tưởng tượng một số tài nguyên (như file JavaScript) cần được tải song song để tăng tốc. Mô hình Queue đơn luồng hiện tại có phù hợp không? Hãy hướng dẫn dùng module `asyncio` hoặc `concurrent.futures` trong Python để giải bài toán tải song song này."*

Bài tập 8: Sinh dãy số nhị phân từ 1 đến N bằng Hàng đợi - Mức độ: Trung bình (Queue)

Bài toán rèn luyện tư duy sinh dữ liệu theo tầng (Level-order / BFS) bằng Queue thay vì dùng phép chia lấy dư thông thường.

Yêu cầu chi tiết:

- Nhập một số nguyên dương n.

- Sinh và in ra tất cả các biểu diễn nhị phân từ số 1 đến số n theo thứ tự tăng dần bằng cách sử dụng **duy nhất một Hàng đợi (Queue)**.
- *Ví dụ:* n = 5 --> Kết quả: `["1", "10", "11", "100", "101"]`.

💡 Gợi ý:

- Khởi tạo Queue với phần tử đầu tiên là chuỗi `"1"`.
- Lặp n lần:
    1. Lấy chuỗi `s` ra khỏi hàng đợi (`popleft()`) và in ra kết quả.
    2. Tạo 2 chuỗi con mới: `s1 = s + "0"` và `s2 = s + "1"`.
    3. Đẩy lần lượt `s1` và `s2` vào lại cuối hàng đợi.

💻 Code mẫu:

```
from collections import deque

def generate_binary_numbers(n: int) -> list[str]:
    if n <= 0:
        return []

    result = []
    q = deque()
    q.append("1")  # Bắt đầu từ 1

    for _ in range(n):
        curr = q.popleft()
        result.append(curr)

        # Sinh hai nhánh con tiếp theo
        q.append(curr + "0")
        q.append(curr + "1")

    return result

def main():
    n = int(input("Nhập số N: "))
    binary_list = generate_binary_numbers(n)
    print(f"Dãy số nhị phân từ 1 đến {n}:")
    print(" -> ".join(binary_list))

if __name__ == "__main__":
    main()
```

🤖 Vận dụng AI:

- **Phân tích thuật toán:** Hỏi AI: *"Vẽ sơ đồ cây nhị phân thể hiện cách thức duyệt theo chiều rộng (BFS) của Queue trong bài toán sinh chuỗi nhị phân này để tôi hình dung rõ cơ chế."*

Bài tập 9: Tính toán biểu thức dạng Postfix (Ký pháp Ba Lan ngược) - Mức độ: Rất khó (Stack)

Đây là một bài toán nâng cao, ứng dụng sâu của Stack trong việc phân tích và tính toán biểu thức toán học.

Yêu cầu chi tiết:

- Viết một hàm nhận đầu vào là một biểu thức Postfix dưới dạng chuỗi. Các toán hạng và toán tử được cách nhau bởi dấu cách (ví dụ: `"5 3 + 2 *"`).
- Hàm này sẽ tính toán và trả về kết quả của biểu thức đó. (Ví dụ trên tương đương $(5 + 3) \times 2 = 16$).
- Hỗ trợ các toán tử `+`, `-`, `*`, `/`.

💡 Gợi ý:

- Tạo một list làm Stack để lưu các toán hạng.
- Tách chuỗi đầu vào thành các token bằng `expression.split(' ')`.
- Duyệt qua từng token:
    - Nếu token là một số (kiểm tra được bằng hàm ép kiểu sang `int`), hãy `append()` vào Stack.
    - Nếu token là một toán tử (`+`, `-`, `*`, `/`):
        - `pop()` hai toán hạng ra khỏi Stack (lưu ý thứ tự: toán hạng thứ hai ra trước, rồi đến toán hạng thứ nhất).
        - Thực hiện phép tính tương ứng. Chú ý phép chia trong Python nên dùng chia nguyên `int(operand1 / operand2)` để giống hành vi chia nguyên của C#.
        - Đẩy kết quả của phép tính trở lại vào Stack.
- Sau khi duyệt hết các token, kết quả cuối cùng chính là phần tử duy nhất còn lại trong Stack. `pop()` nó ra và trả về.

💻 Code mẫu:

```
class PostfixCalculator:
    def evaluate(self, expression: str) -> int:
        operands = []
        tokens = expression.split()

        for token in tokens:
            try:
                # Nếu là số, chuyển thành int và đẩy vào stack
                number = int(token)
                operands.append(number)
            except ValueError:
                # Nếu là toán tử: toán hạng 2 được pop ra trước
                operand2 = operands.pop()
                operand1 = operands.pop()
```

```
                if token == "+":
                    operands.append(operand1 + operand2)
                elif token == "-":
                    operands.append(operand1 - operand2)
                elif token == "*":
                    operands.append(operand1 * operand2)
                elif token == "/":
                    # Truncate towards zero giống cơ chế ép kiểu int trong C#
                    operands.append(int(operand1 / operand2))
                else:
                    raise ValueError(f"Toán tử không hợp lệ: {token}")

        return operands.pop()

# Kiểm thử
calculator = PostfixCalculator()
result = calculator.evaluate("5 3 + 2 *")
print("Kết quả:", result)  # 16
```

🤖 Vận dụng AI:

- **Tìm hiểu nguồn gốc:** Hỏi AI: *"Ký pháp Postfix là gì và tại sao máy tính lại thích xử lý nó hơn ký pháp Infix (dạng a + b) thông thường? Ai là người đã phát minh ra nó?"*
- **Thử thách cuối cùng:** Yêu cầu AI: *"Hãy phác thảo các bước chính và cấu trúc dữ liệu cần dùng để viết một chương trình chuyển đổi một biểu thức Infix đơn giản (ví dụ '5 + 3 * 2') thành biểu thức Postfix ('5 3 2 * +'). (Đây chính là thuật toán Shunting-yard nổi tiếng của Dijkstra và nó cũng dùng Stack)."*

Bài tập 10: Chuyển đổi Infix sang Postfix (Thuật toán Shunting-yard của Dijkstra) - Mức độ: Rất khó (Stack)

Bài tập này bổ trợ trực tiếp cho Bài tập 9 (Tính toán biểu thức Postfix), giúp hoàn thiện một trình biên dịch biểu thức toán học hoàn chỉnh.

Yêu cầu chi tiết:

- Viết hàm chuyển một biểu thức số học dạng Trung tố thông thường (Infix) có dấu ngoặc sang dạng Hậu tố (Postfix).
- Hỗ trợ các toán tử `+`, `-`, `*`, `/`, `^` (lũy thừa) và dấu ngoặc `(`, `)`.
- *Ví dụ:* `"A + B * C"` --> `"A B C * +"`; `"( A + B ) * C"` --> `"A B + C *"`.

💡 Gợi ý:

- Bảng độ ưu tiên toán tử: `+`, `-` (mức 1); `*`, `/` (mức 2); `^` (mức 3).
- Gặp toán hạng (chữ cái/số) --> Đưa ngay vào danh sách kết quả.
- Gặp ngoặc mở `(` --> Push vào Stack.
- Gặp ngoặc đóng `)` --> Lần lượt Pop các toán tử ra danh sách kết quả cho đến khi gặp `(`.

- Gặp toán tử --> Pop các toán tử trên đỉnh Stack có độ ưu tiên lớn hơn hoặc bằng nó ra kết quả, sau đó Push toán tử hiện tại vào Stack.

💻 Code mẫu:

```
def infix_to_postfix(expression: str) -> str:
    precedence = {'+': 1, '-': 1, '*': 2, '/': 2, '^': 3}
    output = []
    stack = []
    tokens = expression.split()

    for token in tokens:
        # Nếu là toán hạng (chữ hoặc số)
        if token.isalnum():
            output.append(token)
        elif token == '(':
            stack.append(token)
        elif token == ')':
            while stack and stack[-1] != '(':
                output.append(stack.pop())
            if stack:
                stack.pop()  # Bỏ dấu '('
        else:
            # Là toán tử
            while (stack and stack[-1] != '(' and
                   precedence.get(stack[-1], 0) >= precedence.get(token, 0)):
                output.append(stack.pop())
            stack.append(token)

    # Đưa các toán tử còn lại trong Stack ra kết quả
    while stack:
        output.append(stack.pop())

    return " ".join(output)

def main():
    exp1 = "( A + B ) * ( C - D )"
    exp2 = "3 + 4 * 2 / ( 1 - 5 )"
    print("Infix 1:  ", exp1)
    print("Postfix 1:", infix_to_postfix(exp1))
    print("\nInfix 2:  ", exp2)
    print("Postfix 2:", infix_to_postfix(exp2))

if __name__ == "__main__":
    main()
```

🤖 Vận dụng AI:

- **Ghép nối toàn diện:** Hỏi AI: *"Hãy kết hợp hàm `infix_to_postfix` của Bài 10 này với hàm `evaluate` ở Bài tập 9 để tạo thành một máy tính cầm tay hoàn chỉnh: người dùng chỉ cần gõ biểu thức tự nhiên như `( 5 + 3 ) * 2` là tính ra ngay đáp số `16`."*